MY SQL
SQL QUERY
PART - 2

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

Rahul Yadav

1. SELECT Statement - Basics

1.1 What is SELECT ?

- SELECT statement is used to retrieve data from a table in MySQL database.
- It is the most commonly used SQL statement.
- We can select one or more columns from a table.
- We can also use expressions, functions, conditions, sorting etc.
- It does not modify data, it only fetches data.

1.2 Basic Syntax

```
SELECT column1, column2, ...
FROM table_name;
```

Explanation:
- SELECT - keyword to fetch data
- column1, column2 - columns to retrieve
- FROM - keyword to specify table
- table_name - name of the table

1.3 Select All Columns

```
SELECT *
FROM employees;
```

- * (asterisk) selects all columns.
- Useful when we want to see complete data from a table.

1.4 Select Specific Columns

```
SELECT id, name, department, salary
FROM employees;
```

- We can select required columns only.
- This improves readability and performance for large tables.

1.5 Example with Output

```
SELECT id, name, department
FROM employees;
```

id	name	department
1	Rahul	IT
2	Anita	HR
3	Suresh	Finance
4	Pooja	IT
5	Karan	Operations

Key Points:
- SELECT is case-insensitive (select = SELECT).
- Column names are usually case-insensitive.
- Use comma (,) to select multiple columns.
- Use * to select all columns.
- Always specify required columns in production (avoid SELECT *).
- Use meaningful column names.
- We can also use expressions, alias, functions and conditions with SELECT (covered in next pages).

Rahul Yadav

2. WHERE Clause - Filtering Data

2.1 What is WHERE Clause ?

- WHERE clause is used to filter records from a table based on a condition.
- It returns only the rows that satisfy the given condition.
- We can use comparison operators, logical operators and expressions.
- It is one of the most important clauses in SQL.

2.2 Basic Syntax

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Explanation:

- SELECT - columns to retrieve
- FROM - table name
- WHERE - condition to filter rows
- condition - any valid expression (e.g. id > 10, name = 'Rahul')

2.3 Comparison Operators

Operator	Description	Operator	Description
=	Equal to	<> or !=	Not equal to
>	Greater than	>=	Greater than or equal to
<	Less than	<=	Less than or equal to

2.4 Examples

1. Filter by single condition

```
SELECT *
FROM employees
WHERE department = 'IT';
```

- Returns all employees from IT department.

2. Filter using multiple conditions

```
SELECT id, name, salary
FROM employees
WHERE department = 'IT'
  AND salary > 50000;
```

- Returns employees from IT department with salary greater than 50000.

3. Using comparison operator

```
SELECT name, salary
FROM employees
WHERE salary >= 60000;
```

- Returns employees having salary greater than or equal to 60000.

4. Using not equal to

```
SELECT *
FROM employees
WHERE department <> 'HR';
```

- Returns employees who are not from HR department.

5. Using multiple conditions (OR)

```
SELECT *
FROM employees
WHERE department = 'IT'
  OR department = 'Finance';
```

- Returns employees from IT or Finance department.

6. Using numeric condition

```
SELECT name, age
FROM employees
WHERE age > 25 AND age <= 35;
```

- Returns employees with age between 26 and 35.

7. Using string condition

```
SELECT *
FROM employees
WHERE name = 'Rahul';
```

- Returns the employee whose name is Rahul.

8. Using date condition

```
SELECT *
FROM employees
WHERE joining_date >= '2023-01-01';
```

- Returns employees joined on or after 1st Jan 2023.

Rahul Yadav

3. AND, OR, NOT Operators

3.1 Introduction

- AND, OR and NOT are logical operators used with WHERE clause.
- They are used to combine multiple conditions.
- They help to filter data based on more than one condition.
- We can also use parentheses ( ) to control the order of evaluation.

3.2 AND Operator

- AND operator returns records only when all conditions are true.
- Both conditions must be satisfied.

```
SELECT column1, column2
FROM table_name
WHERE condition1 AND condition2;
```

Example:

```
SELECT id, name, salary
FROM employees
WHERE department = 'IT'
      AND salary > 50000;
```

3.3 OR Operator

- OR operator returns records when at least one condition is true.
- If any one condition is satisfied, the row will be returned.

```
SELECT column1, column2
FROM table_name
WHERE condition1 OR condition2;
```

Example:

```
SELECT id, name, department
FROM employees
WHERE department = 'IT'
      OR department = 'Finance';
```

3.4 NOT Operator

- NOT operator is used to negate a condition.
- It returns records that do NOT satisfy the given condition.

```
SELECT column1, column2
FROM table_name
WHERE NOT condition;
```

Example:

```
SELECT id, name, department
FROM employees
WHERE NOT department = 'HR';
```

3.5 Combining Operators

- We can use AND, OR and NOT together in a single query.
- Use parentheses ( ) to define the order of operations and avoid confusion.

```
SELECT id, name, salary, department
FROM employees
WHERE (department = 'IT' OR department = 'Finance')
      AND salary > 50000
      AND NOT name = 'Rahul';
```

Explanation:

- Selects employees from IT or Finance department.
- Salary should be greater than 50000.
- Name should not be Rahul.
- Parentheses ensure OR condition is evaluated first.

3.6 Truth Table

A	B	A AND B	A OR B	NOT A	NOT B
True	True	True	True	False	False
True	False	False	True	False	True
False	True	False	True	True	False
False	False	False	False	True	True

Rahul Yadav

4. IN, NOT IN, BETWEEN, LIKE and NULL Handling

4.1 IN Operator

- IN operator is used to match a value with
- a list of values.
- It is a shorthand for multiple OR conditions.
- We can use it with numbers, strings or subqueries.

Example:

```
SELECT id, name, department
FROM employees
WHERE department IN ('IT', 'HR', 'Finance');
```

Equivalent using OR:

```
SELECT id, name, department
FROM employees
WHERE department = 'IT'
      OR department = 'HR'
      OR department = 'Finance';
```

4.2 NOT IN Operator

- NOT IN operator is used to exclude a list of values.
- It returns rows where the column value is not in the given list.

Example:

```
SELECT id, name, department
FROM employees
WHERE department NOT IN ('IT', 'HR');
```

Note:

- If the list contains NULL, the result may be unexpected.
- Use NOT EXISTS instead of NOT IN when working with subqueries and NULL.

4.3 BETWEEN Operator

- BETWEEN operator is used to filter values within a range (INCLUSIVE).
- It works for numbers, dates and strings (alphabetical range).

Example (Numbers):

```
SELECT id, name, salary
FROM employees
WHERE salary BETWEEN 50000 AND 100000;
```

Example (Dates):

```
SELECT id, name, joining_date
FROM employees
WHERE joining_date BETWEEN '2023-01-01'
      AND '2023-12-31';
```

4.4 LIKE Operator

- LIKE operator is used for pattern matching with string values.
- We can use % (any number of characters) and _ (single character).

Examples:

```
SELECT * FROM employees WHERE name LIKE 'Rah%';  -- starts with Rah
SELECT * FROM employees WHERE name LIKE '%ul';   -- ends with ul
SELECT * FROM employees WHERE name LIKE '%ah%';  -- contains ah
SELECT * FROM employees WHERE name LIKE 'R_h_l'; -- R?hul (single char)
```

Wildcards in LIKE:

- % - any number of characters
- _ - single character

Example:

```
LIKE 'R%'
LIKE '%ul'
LIKE '_a%'
```

4.5 NULL Handling

- NULL represents missing or unknown value.
- We cannot use = or <> to compare with NULL.
- We must use IS NULL or IS NOT NULL.

Example:

```
SELECT id, name, manager_id
FROM employees
WHERE manager_id IS NULL;
```

Comparison with NULL

Query	Result
= NULL	Not valid (returns no rows)
<> NULL	Not valid (returns no rows)
IS NULL	Correct way to check NULL
IS NOT NULL	Correct way to check not NULL

Example (Not NULL):

```
SELECT id, name
FROM employees
WHERE manager_id IS NOT NULL;
```

Rahul Yadav

5. ORDER BY, LIMIT and OFFSET

5.1 ORDER BY Clause

- ORDER BY is used to sort the result set.
- We can sort in ascending (ASC) or descending (DESC) order.
- We can sort by one or multiple columns.
- By default, the order is ASC (ascending).

Example:

```
SELECT id, name, salary
FROM employees
ORDER BY salary ASC;
```

Example (Descending):

```
SELECT id, name, salary
FROM employees
ORDER BY salary DESC;
```

5.2 Order by Multiple Columns

- We can sort by multiple columns.
- The next column is used to break the tie when values are same.
- We can specify different order (ASC/DESC) for each column.

Example:

```
SELECT id, name, department, salary
FROM employees
ORDER BY department ASC, salary DESC;
```

Sample Result:

id	name	department	salary
4	Pooja	Finance	80000
2	Anita	Finance	60000
1	Rahul	IT	90000
3	Suresh	IT	70000

5.3 LIMIT Clause

- LIMIT is used to return a specific number of rows.
- It is useful when we only need top N records.

Example:

```
SELECT id, name, salary
FROM employees
ORDER BY salary DESC
LIMIT 5;
```

This will return:

- Top 5 employees having highest salary.
- If you need only 1 record, use LIMIT 1.

5.4 OFFSET Clause

- OFFSET is used to skip a specific number of rows.
- It is commonly used with LIMIT for pagination.

Example:

```
SELECT id, name, salary
FROM employees
ORDER BY id ASC
LIMIT 5 OFFSET 5;
```

Explanation:

- This will skip first 5 rows and return next 5 rows.
- Useful for pagination (page 2, page 3, etc).
- Page 1: LIMIT 5 OFFSET 0
- Page 2: LIMIT 5 OFFSET 5
- Page 3: LIMIT 5 OFFSET 10

5.5 Real World Use Case – Pagination

- Pagination helps to show data in small chunks (e.g. 10 records per page).

Example Query:

```
SELECT id, name, email
FROM employees
ORDER BY id ASC
LIMIT 10 OFFSET 20;
```

Explanation:

- Shows 10 records per page.
- Skips first 20 records.
- This will return records from 21 to 30.
- Useful in web applications and APIs.

Rahul Yadav

6. GROUP BY and HAVING Clause

6.1 What is GROUP BY ?

- GROUP BY is used to group rows that have the same values.
- It is mainly used with aggregate functions like COUNT, SUM, AVG, MIN, MAX.
- It returns one row for each group.
- It is useful for generating summaries and reports.

6.2 Basic Syntax

```
SELECT column1, aggregate_function(column2)
FROM table_name
GROUP BY column1;
```

Explanation:

- column1 - column to group data
- aggregate_function() - function like COUNT, SUM, AVG, etc.
- GROUP BY - groups rows with same value

6.3 Example - Count Employees by Department

```
SELECT department, COUNT(*) as total_count
FROM employees
GROUP BY department;
```

Sample Result:

department	total_count
Finance	3
HR	2
IT	4
Operations	1

6.4 Example - Calculate Average Salary by Department

```
SELECT department, AVG(salary) as avg_salary
FROM employees
GROUP BY department;
```

Sample Result:

department	avg_salary
Finance	65000
HR	55000
IT	80000
Operations	50000

6.5 What is HAVING ?

- HAVING is used to filter the grouped data.
- It is similar to WHERE but used with GROUP BY.
- We cannot use aggregate functions in WHERE, so we use HAVING.
- It is applied after grouping.

6.6 Syntax with HAVING

```
SELECT column1, aggregate_function(column2)
FROM table_name
GROUP BY column1
HAVING condition;
```

Explanation:

- Groups the data first (GROUP BY).
- Then applies condition on grouped result using HAVING.
- We can use aggregate functions in HAVING.

6.7 Example - Departments having more than 2 employees

```
SELECT department, COUNT(*) as total_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 2;
```

Sample Result:

department	total_count
Finance	3
IT	4

Rahul Yadav

7. DISTINCT, COUNT, SUM, AVG, MIN, MAX

7.1 DISTINCT Keyword

- DISTINCT is used to remove duplicate values from the result set.
- It returns only unique values.
- It can be used with one or multiple columns.
- Useful for finding unique departments, unique cities, etc.

Example:

```
SELECT DISTINCT department
FROM   employees
GRDER BY department;
```

Sample Result:

department
Finance
HR
IT
Operations

7.2 COUNT() Function

- COUNT() is used to count the number of rows.
- COUNT(*) counts all rows (including NULL).
- COUNT(column) counts only non-NULL values.
- Useful for reports, dashboards and analytics.

Examples:

```
SELECT COUNT(*) FROM employees;  -- total rows
SELECT COUNT(salary) FROM employees; -- non NULL salary
SELECT department, COUNT(*)
FROM   employees
GROUP BY department;
```

Sample Result:

department	total_count
Finance	3
HR	2
IT	4
Operations	1

7.3 SUM() Function

- SUM() is used to calculate the total sum of a numeric column.
- It ignores NULL values.
- Useful for total salary, total sales, total amount, etc.

Example:

```
SELECT department, SUM(salary) AS total_salary
FROM   employees
GRDUP BY department;
```

Sample Result:

department	total_salary
Finance	195000
HR	110000
IT	320000
Operations	70000

7.4 AVG() Function

- AVG() is used to calculate the average of a numeric column.
- It ignores NULL values.
- Useful for average salary, average order amount, etc.

Example:

```
SELECT department, AVG(salary) AS avg_salary
FROM   employees
GROUP BY department;
```

Sample Result:

department	avg_salary
Finance	65000
HR	55000
IT	80000
Operations	70000

7.5 MIN() and MAX() Function

- MIN() returns the smallest value.
- MAX() returns the largest value.
- Useful for finding minimum and maximum salary, date, id, etc.

Example:

```
SELECT MIN(salary) AS min_salary,
       MAX(salary) AS max_salary
FROM   employees;
```

Sample Result:

min_salary	max_salary
40000	120000

Rahul Yadav

10. UNION, UNION ALL and SELF JOIN

10.1 UNION vs UNION ALL

- UNION is used to combine the result sets of two or more SELECT statements.
- UNION removes duplicate records from the final result.
- UNION ALL returns all records including duplicates.
- Both SELECT statements must have the same number of columns and compatible data types.

Example: UNION (removes duplicates)

```
SELECT name, department
FROM employees
UNION
SELECT name, department
FROM contractors;
```

Result:

name	department
Rahul	IT
Anita	Finance
Suresh	IT
Pooja	Operations

Example: UNION ALL (keeps duplicates)

```
SELECT name, department
FROM employees
UNION ALL
SELECT name, department
FROM contractors;
```

Result:

name	department
Rahul	IT
Anita	Finance
Suresh	IT
Pooja	Operations
Rahul	IT
Anita	Finance

10.2 SELF JOIN

- A table is joined with itself.
- Used to compare rows within the same table.
- Common use cases: finding employees and their managers, hierarchical data, comparing values, etc.

Example:

```
SELECT e.name AS employee_name,
       m.name AS manager_name
FROM   employees e
LEFT JOIN employees m
       ON e.manager_id = m.emp_id;
```

Result:

employee_name	manager_name
Rahul	Suresh
Anita	Rahul
Suresh	NULL
Pooja	Rahul

10.3 Multiple Joins

- We can join more than two tables.
- Useful when data is spread across multiple tables.
- Helps to combine related data in a single query.

Example:

```
SELECT e.name, d.dept_name, l.location_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
JOIN locations l ON d.location_id = l.location_id;
```

Result:

name	dept_name	location_name
Rahul	IT	Mumbai
Anita	Finance	Delhi
Suresh	IT	Bengaluru
Pooja	Operations	Pune

10.4 Cross Join (Cartesian Product)

- CROSS JOIN returns the Cartesian product (all possible combinations) of rows from both tables.
- If table A has m rows and table B has n rows, result will have m × n rows.
- It is rarely used in real applications.

Example:

```
SELECT e.name, d.dept_name
FROM   employees e
CROSS JOIN departments d;
```

Result: (If 2 employees × 3 departments = 6 rows)

name	dept_name
Rahul	IT
Rahul	Finance
Rahul	Operations
Anita	IT
Anita	Finance
Anita	Operations

Rahul Yadav

8. WHERE, AND, OR, NOT with Functions

8.1 WHERE Clause with Functions

- We can use functions inside WHERE clause.
- Useful for filtering data based on calculations, string operations, dates, etc.
- Common functions: UPPER(), LOWER(), LENGTH(), YEAR(), MONTH(), etc.

Example 1: Filter by string function

```
SELECT id, name, department
FROM employees
WHERE UPPER(department) = 'IT';
```

Result:

id	name	department
1	Rahul	IT
3	Suresh	IT

Example 2: Filter by date function

```
SELECT id, name, joining_date
FROM employees
WHERE YEAR(joining_date) = 2023;
```

Result:

id	name	joining_date
1	Rahul	2023-05-10
4	Pooja	2023-11-20

8.2 AND, OR, NOT with Functions

- We can combine logical operators (AND, OR, NOT) with functions.
- It helps to apply multiple conditions and filter data more precisely.
- Parentheses ( ) are used to control the order of evaluation.

Example 1: Using AND

```
SELECT id, name, salary
FROM employees
WHERE department = 'IT'
      AND salary > 50000;
```

Result:

id	name	salary
1	Rahul	90000
3	Suresh	70000

Example 2: Using OR

```
SELECT id, name, department
FROM employees
WHERE department = 'IT'
      OR department = 'Finance';
```

Result:

id	name	department
1	Rahul	IT
2	Anita	Finance
3	Suresh	IT

Example 3: Using NOT with Function

```
SELECT id, name, department
FROM employees
WHERE NOT UPPER(department) = 'HR';
```

Result:

id	name	department
1	Rahul	IT
2	Anita	Finance
3	Suresh	IT
4	Pooja	Operations

8.3 Combining Multiple Conditions

- We can use multiple functions with AND, OR and NOT together.
- Useful for complex filtering.

Example:

```
SELECT id, name, salary, department
FROM employees
WHERE UPPER(department) = 'IT'
      AND salary > 50000
      AND LENGTH(name) > 4;
```

Result:

id	name	salary	department
1	Rahul	90000	IT
3	Suresh	70000	IT

Rahul Yadav

9. JOINS in MySQL

9.1 What is JOIN ?

- JOIN is used to combine rows from two or more tables based on a related column.
- It helps to fetch data from multiple tables in a single query.
- Most common joins: INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN (using UNION), SELF JOIN and CROSS JOIN.

9.2 Types of JOINs

- INNER JOIN - returns matching records from both tables.
- LEFT JOIN - returns all records from left table and matched records from right table.
- RIGHT JOIN - returns all records from right table and matched records from left table.
- FULL JOIN - returns all records from both tables (MySQL uses UNION of LEFT and RIGHT JOIN).
- CROSS JOIN - returns Cartesian product (all combinations).
- SELF JOIN - join a table with itself.

Table Used in Examples:

employees (emp_id, name, dept_id)

emp_id	name	dept_id
1	Rahul	1
2	Anita	2
3	Suresh	1
4	Pooja	3

departments (dept_id, dept_name)

dept_id	dept_name
1	IT
2	Finance
3	Operations

9.3 INNER JOIN (Matching records only)

Query:

```
SELECT e.emp_id, e.name, d.dept_name
FROM employees e
INNER JOIN departments d
    ON e.dept_id = d.dept_id;
```

Result:

emp_id	name	dept_name
1	Rahul	IT
3	Suresh	IT
2	Anita	Finance
4	Pooja	Operations

9.4 LEFT JOIN (All records from left table)

Query:

```
SELECT e.emp_id, e.name, d.dept_name
FROM employees e
LEFT JOIN departments d
    ON e.dept_id = d.dept_id;
```

Result:

emp_id	name	dept_name
1	Rahul	IT
2	Anita	Finance
3	Suresh	IT
4	Pooja	Operations

9.5 RIGHT JOIN (All records from right table)

Query:

```
SELECT e.emp_id, e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d
    ON e.dept_id = d.dept_id;
```

Result:

emp_id	name	dept_name
1	Rahul	IT
3	Suresh	IT
2	Anita	Finance
4	Pooja	Operations

9.6 FULL JOIN (All records from both tables)

Query (Using UNION):

```
SELECT e.emp_id, e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
UNION
SELECT e.emp_id, e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

Result:

emp_id	name	dept_name
1	Rahul	IT
2	Anita	Finance
3	Suresh	IT
4	Pooja	Operations

Rahul Yadav

11. SUBQUERY in MySQL

11.1 What is Subquery ?

- A subquery is a query inside another query.
- It is used to get data that will be used in the main query.
- Subquery can be used in SELECT, FROM, WHERE and HAVING clauses.
- It helps to solve complex queries in a simple way.

11.2 Types of Subqueries

- Single Row Subquery - returns only one row.
- Multiple Row Subquery - returns multiple rows.
- Multiple Column Subquery - returns multiple columns.
- Correlated Subquery - depends on the outer query for each row.

11.3 Single Row Subquery

- Returns only one row (one value).
- Used with operators like =, >, <, >=, <=, <>.

Example:

```
SELECT name, salary
FROM   employees
WHERE  salary > (SELECT AVG(salary)
                 FROM   employees);
```

Result:

name	salary
Rahul	90000
Anita	80000
Suresh	75000

11.4 Multiple Row Subquery

- Returns more than one row (one column).
- Used with IN, NOT IN, ANY, ALL.

Example:

```
SELECT name, department
FROM   employees
WHERE  department IN (SELECT dept_name
                      FROM   departments
                      WHERE  location = 'Mumbai');
```

Result:

name	department
Rahul	IT
Suresh	IT
Pooja	Operations

11.5 Multiple Column Subquery

- Returns more than one column.
- Used to compare multiple columns at once.

Example:

```
SELECT name, department, salary
FROM   employees
WHERE  (department, salary) IN (SELECT department, MAX(salary)
                                FROM   employees
                                GROUP BY department);
```

Result:

name	department	salary
Rahul	IT	90000
Anita	Finance	85000
Pooja	Operations	70000

11.6 Correlated Subquery

- Depends on the outer query for each row.
- Executes once for each row of the outer query.
- Useful for row-by-row comparison.

Example:

```
SELECT e.name, e.salary
FROM   employees e
WHERE  e.salary > (SELECT AVG(salary)
                   FROM   employees e2
                   WHERE  e2.department = e.department);
```

Result:

name	salary
Rahul	90000
Suresh	75000
Pooja	70000

Rahul Yadav

12. GROUP BY and HAVING

12.1 What is GROUP BY ?

- GROUP BY is used to group rows based on one or more columns.
- It is used with aggregate functions like COUNT(), SUM(), AVG(), MIN(), MAX().
- It helps to perform calculations on each group of rows.
- Used to get summarized data from large datasets.

Example 1: Count employees per department

```
SELECT  department, COUNT(*) AS total_employees
FROM    employees
GROUP   BY department;
```

Result:

department	total_employees
IT	3
Finance	2
Operations	1

12.2 GROUP BY with SUM(), AVG()

- We can use aggregate functions with GROUP BY.
- Useful for total salary, average salary, etc.
- We can group by single or multiple columns.

Example: Find total and average salary per department

```
SELECT  department,
        SUM(salary) AS total_salary,
        AVG(salary) AS avg_salary
FROM    employees
GROUP   BY department;
```

Result:

department	total_salary	avg_salary
IT	270000	90000
Finance	160000	80000
Operations	70000	70000

12.3 HAVING Clause

- HAVING is used to filter the grouped results.
- It works like WHERE but is used after GROUP BY.
- We can use aggregate functions in HAVING.

Example: Departments with more than 1 employee

```
SELECT  department, COUNT(*) AS total_employees
FROM    employees
GROUP   BY department
HAVING  COUNT(*) > 1;
```

Result:

department	total_employees
IT	3
Finance	2

12.4 GROUP BY with Multiple Columns

- We can group by multiple columns to get more detailed results.
- Useful when we need a summary based on two or more fields.

Example: Count employees per department and location

```
SELECT  department, location, COUNT(*) AS total
FROM    employees
GROUP   BY department, location;
```

Result:

department	location	total
IT	Mumbai	2
IT	Delhi	1
Finance	Mumbai	1
Finance	Pune	1
Operations	Bengaluru	1

12.5 HAVING with Aggregate Conditions

- We can apply conditions on aggregate values (SUM, AVG, COUNT, etc).
- Useful for filtering groups based on totals.

Example: Departments with total salary greater than 150000

```
SELECT  department, SUM(salary) AS total_salary
FROM    employees
GROUP   BY department
HAVING  SUM(salary) > 150000;
```

Result:

department	total_salary
IT	270000
Finance	160000

Rahul Yadav

13. ORDER BY and LIMIT

13.1 ORDER BY Clause

- ORDER BY is used to sort the result set in ascending (ASC) or descending (DESC) order.
- By default, it sorts in ascending order (ASC).
- We can sort by one or multiple columns.
- Commonly used with SELECT, GROUP BY, etc.

Example 1: Sort employees by salary (Ascending)

```
SELECT id, name, salary
FROM employees
ORDER BY salary ASC;
```

Result:

id	name	salary
3	Suresh	70000
2	Anita	80000
1	Rahul	90000
4	Pooja	95000

Example 2: Sort by salary (Descending)

```
SELECT id, name, salary
FROM employees
ORDER BY salary DESC;
```

Result:

id	name	salary
4	Pooja	95000
1	Rahul	90000
2	Anita	80000
3	Suresh	70000

Example 3: Sort by multiple columns

```
SELECT id, name, department, salary
FROM employees
ORDER BY department ASC, salary DESC;
```

Result:

id	name	department	salary
1	Rahul	Finance	90000
2	Anita	Finance	80000
3	Suresh	IT	70000
4	Pooja	Operations	95000

13.2 LIMIT Clause

- LIMIT is used to restrict the number of rows returned by a query.
- It is useful to get top N records.
- Often used with ORDER BY.

Example 1: Get top 2 highest paid employees

```
SELECT id, name, salary
FROM employees
ORDER BY salary DESC
LIMIT 2;
```

Result:

id	name	salary
4	Pooja	95000
1	Rahul	90000

Example 2: Get top 3 employees

```
SELECT id, name, department, salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

Result:

id	name	department	salary
4	Pooja	Operations	95000
1	Rahul	Finance	90000
2	Anita	Finance	80000

13.3 LIMIT with OFFSET

- OFFSET is used to skip a specific number of rows.
- Useful for pagination (e.g., page 2, page 3, etc.).
- Syntax: LIMIT number_of_rows OFFSET starting_position;

Example: Get next 2 employees (skip first 2)

```
SELECT id, name, salary
FROM employees
ORDER BY salary DESC
LIMIT 2 OFFSET 2;
```

Result:

id	name	salary
2	Anita	80000
3	Suresh	70000

Rahul Yadav

14. SQL Window Functions

14.1 What are Window Functions ?

- Window functions perform calculations across a set of table rows related to the current row.
- They do not collapse rows like GROUP BY (rows remain the same).
- They use OVER() clause to define a window (partition and ordering).
- Common use cases: running total, rank, row number, moving average, comparing with previous/next row.

14.2 OVER Clause Syntax

```
SELECT column1, column2,
       function() OVER (
           [PARTITION BY column]
           [ORDER BY column]
           [ROWS | RANGE frame_clause]
       ) AS alias
FROM table_name;
```

Key Components:

- OVER() - defines the window for calculation.
- PARTITION BY - divides the result set into partitions (like GROUP BY but rows are not collapsed).
- ORDER BY - defines the order within each partition.
- ROWS/RANGE - defines the frame of rows to consider (optional).

14.3 ROW_NUMBER() - Unique sequential number

- Assigns a unique number to each row within a partition.
- Useful for pagination, finding top N records, removing duplicates.

Example: Get row number for each employee by department

```
SELECT id, name, department, salary,
       ROW_NUMBER() OVER (
         PARTITION BY department
         ORDER BY salary DESC
       ) AS row_num
FROM employees;
```

Result:

id	name	department	salary	row_num
1	Rahul	IT	90000	1
3	Suresh	IT	75000	2
2	Anita	Finance	80000	1
4	Pooja	Finance	70000	2

14.4 RANK() and DENSE_RANK() - Ranking rows

- RANK() gives the same rank for same values and skips the next rank.
- DENSE_RANK() gives the same rank for same values without skipping the next rank.

Example: Rank employees by salary within department

```
SELECT id, name, department, salary,
       RANK() OVER (PARTITION BY department
                    ORDER BY salary DESC) AS rank_num,
       DENSE_RANK() OVER (PARTITION BY department
                    ORDER BY salary DESC) AS dense_rank
FROM employees;
```

Result:

id	name	department	salary	rank_num	dense_rank
1	Rahul	IT	90000	1	1
3	Suresh	IT	75000	2	2
5	Mohit	IT	75000	2	2
2	Anita	Finance	80000	1	1
4	Pooja	Finance	70000	2	2

14.5 Other Useful Window Functions

Function	Description	Example
ROW_NUMBER()	Unique number for each row	ROW_NUMBER() OVER (ORDER BY salary)
RANK()	Rank with gaps for duplicates	RANK() OVER (ORDER BY salary)
DENSE_RANK()	Rank without gaps for duplicates	DENSE_RANK() OVER (ORDER BY salary)
NTILE(n)	Divides rows into n groups	NTILE(4) OVER (ORDER BY salary)
LAG()	Access previous row value	LAG(salary) OVER (ORDER BY id)
LEAD()	Access next row value	LEAD(salary) OVER (ORDER BY id)
FIRST_VALUE()	First value in window	FIRST_VALUE(salary) OVER (PARTITION BY dept)
LAST_VALUE()	Last value in window	LAST_VALUE(salary) OVER (PARTITION BY dept)
SUM() OVER()	Running total / partition total	SUM(salary) OVER (PARTITION BY dept)
AVG() OVER()	Running average / partition average	AVG(salary) OVER (PARTITION BY dept)

Rahul Yadav

15. Common Table Expressions (CTEs) and Recursive Queries in MySQL

15.1 What is a CTE ?

- A Common Table Expression (CTE) is a temporary result set that can be used within a SELECT, INSERT, UPDATE or DELETE statement.
- It makes complex queries more readable and modular.
- CTEs exist only for the duration of the query and are not stored permanently.
- Introduced in MySQL 8.0+.

15.2 Basic CTE Syntax

```
WITH cte_name AS (
    SELECT column1, column2
    FROM table_name
    WHERE condition
)
SELECT * FROM cte_name;
```

Key Points:

- WITH keyword is used to define a CTE.
- A CTE can be referenced like a normal table.
- We can define multiple CTEs in a single query.
- CTEs help in breaking complex queries into smaller parts.

15.3 Example 1: Find employees with salary greater than department average using CTE

```
WITH dept_avg AS (
    SELECT department, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
)
SELECT e.id, e.name, e.department, e.salary, d.avg_salary
FROM employees e
JOIN dept_avg d ON e.department = d.department
WHERE e.salary > d.avg_salary;
```

Result:

id	name	department	salary	avg_salary
1	Rahul	IT	90000	75000
4	Pooja	IT	85000	75000
6	Mohit	Finance	90000	80000

15.4 Multiple CTEs Example: Department wise top earner

```
WITH dept_avg AS (
    SELECT department, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
),
dept_top AS (
    SELECT department, MAX(salary) AS max_salary
    FROM employees
    GROUP BY department
)
SELECT e.id, e.name, e.department, e.salary, d.avg_salary, t.max_salary
FROM employees e
JOIN dept_avg d ON e.department = d.department
JOIN dept_top t ON e.department = t.department
WHERE e.salary = t.max_salary;
```

Result:

id	name	department	salary	avg_salary	max_salary
1	Rahul	IT	90000	75000	90000
5	Sunita	Finance	95000	80000	95000
8	Vikram	Operations	88000	70000	88000

15.5 Recursive CTE - What is it ?

- A Recursive CTE is used to solve hierarchical or tree-structured data problems.
- It references itself in its own definition.
- Useful for employee hierarchies, category trees, organization structure, etc.
- Consists of two parts: Anchor Query (base case) and Recursive Query (recursive case).

15.6 Example 2: Employee Hierarchy using Recursive CTE

```
WITH RECURSIVE emp_hierarchy AS (
    -- Anchor Query (top level manager)
    SELECT id, name, manager_id, 1 AS level
    FROM employees
    WHERE manager_id IS NULL
    UNION ALL
    -- Recursive Query (get subordinates)
    SELECT e.id, e.name, e.manager_id, h.level + 1
    FROM employees e
    JOIN emp_hierarchy h ON e.manager_id = h.id
)
SELECT id, name, manager_id, level
FROM emp_hierarchy
ORDER BY level, id;
```

Result:

id	name	manager_id	level
1	Rajesh	NULL	1
2	Suresh	1	2
3	Anita	1	2
4	Rahul	2	3
5	Pooja	2	3
6	Mohit	3	3
7	Neha	3	3
8	Vikram	5	4

Rahul Yadav

16. JOINS in MySQL

16.1 What are Joins ?

- JOIN is used to combine rows from two or more tables based on a related column.
- It allows us to get data from multiple tables in a single query.
- Different types of joins are used based on the requirement.
- Common types: INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, CROSS JOIN, SELF JOIN.

16.2 INNER JOIN

- Returns only the matching rows from both tables.
- Rows must satisfy the join condition.

Example: Get employee details with department name (only matching records)

```
SELECT e.id, e.name, d.department
FROM   employees e
INNER JOIN departments d
   ON e.department_id = d.id;
```

Result:

id	name	department
1	Rahul	IT
2	Anita	Finance
3	Suresh	Operations

16.3 LEFT JOIN

- Returns all rows from the left table and matching rows from the right table.
- If no match is found, NULL is returned for the right table columns.

Example: Get all employees with department name (including employees without department)

```
SELECT e.id, e.name, d.department
FROM   employees e
LEFT  JOIN departments d
    ON e.department_id = d.id;
```

Result:

id	name	department
1	Rahul	IT
2	Anita	Finance
3	Suresh	Operations
4	Pooja	NULL

16.4 RIGHT JOIN

- Returns all rows from the right table and matching rows from the left table.
- If no match is found, NULL is returned for the left table columns.

Example: Get all departments with employee details (including departments without employees)

```
SELECT e.id, e.name, d.department
FROM   employees e
RIGHT JOIN departments d
    ON e.department_id = d.id;
```

Result:

id	name	department
1	Rahul	IT
2	Anita	Finance
3	NULL	HR

16.5 FULL JOIN (MySQL does not support directly)

- Returns all rows when there is a match in either left or right table.
- MySQL does not support FULL JOIN directly.
- We can achieve it using UNION of LEFT JOIN and RIGHT JOIN.

Example: Simulate FULL JOIN

```
SELECT e.id, e.name, d.department
FROM   employees e
LEFT  JOIN departments d
    ON e.department_id = d.id
UNION
SELECT e.id, e.name, d.department
FROM   employees e
RIGHT JOIN departments d
    ON e.department_id = d.id;
```

Result:

id	name	department
1	Rahul	IT
2	Anita	Finance
3	Suresh	Operations
4	Pooja	NULL
5	NULL	HR

16.6 CROSS JOIN

- Returns the Cartesian product (all possible combinations) of rows from both tables.
- It does not require a join condition.
- Useful when we need all combinations (use with caution for large tables).

Example: Get all combinations of employees and departments

```
SELECT e.name, d.department
FROM   employees e
CROSS JOIN departments d;
```

Result:

name	department
Rahul	IT
Rahul	Finance
Anita	IT
Anita	Finance

Rahul Yadav

17. Subqueries in MySQL

17.1 What is a Subquery ?

- A subquery is a query inside another query.
- It is used to get data that is used in the main query.
- Subqueries can be used in SELECT, FROM, WHERE and HAVING clauses.
- It helps to break complex queries into smaller and readable parts.

17.2 Types of Subqueries

- Single Row Subquery – returns one row and one column.
- Multiple Row Subquery – returns multiple rows.
- Multiple Column Subquery – returns multiple columns.
- Correlated Subquery – inner query depends on the outer query.
- Non-Correlated Subquery – inner query runs independently.

17.3 Single Row Subquery

- Returns only one row and one column.
- Used with operators like =, >, <, >=, <=, <>.

Example: Get employees with salary greater than the average salary

```
SELECT id, name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Result:

id	name	salary
1	Rahul	90000
3	Suresh	75000
5	Mohit	80000

17.4 Multiple Row Subquery

- Returns multiple rows (one column).
- Used with operators like IN, NOT IN, ANY, ALL.

Example: Get employees who work in departments located in Mumbai

```
SELECT id, name, department_id
FROM employees
WHERE department_id IN (
        SELECT id
        FROM departments
        WHERE location = 'Mumbai'
);
```

Result:

id	name	department_id
1	Rahul	1
4	Pooja	2
6	Mohit	2

17.5 Multiple Column Subquery

- Returns multiple columns.
- Used with row constructors (column1, column2) IN (subquery).

Example: Get employees with same (name, department_id) as in another table

```
SELECT id, name, department_id
FROM employees
WHERE (name, department_id) IN (
        SELECT name, department_id
        FROM temp_employees
);
```

Result:

id	name	department_id
1	Rahul	1
3	Suresh	2

17.6 Correlated Subquery

- Inner query depends on the outer query.
- Executed for each row of the outer query.

Example: Get employees who earn more than the average salary of their department

```
SELECT e.id, e.name, e.salary, e.department_id
FROM employees e
WHERE e.salary > (
        SELECT AVG(salary)
        FROM employees
        WHERE department_id = e.department_id
);
```

Result:

id	name	salary	department_id
1	Rahul	90000	1
5	Mohit	80000	2

17.7 Subquery in FROM Clause (Derived Table)

- We can use a subquery in FROM clause as a temporary result set.
- Useful for further filtering and aggregation.

Example: Get departments wise average salary using subquery

```
SELECT dept_id, avg_salary
FROM (
    SELECT department_id AS dept_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department_id
) AS dept_avg;
```

Result:

dept_id	avg_salary
1	90000
2	80000
3	70000

Rahul Yadav

18. Indexes in MySQL

18.1 What is an Index ?

- An index is a data structure that helps to speed up data retrieval operations on a table.
- It works like the index of a book, allowing MySQL to find rows quickly without scanning the entire table.
- Indexes improve the performance of SELECT queries, especially on large tables.
- **Trade-off**: Indexes take extra storage space and can slow down INSERT, UPDATE and DELETE operations.

18.2 Types of Indexes

- Primary Index - automatically created for PRIMARY KEY (unique).
- Unique Index - ensures all values in the index are unique.
- Normal Index - improves search performance (duplicates allowed).
- Composite Index - index on multiple columns.
- Fulltext Index - used for text search (mainly for CHAR, VARCHAR, TEXT).
- Spatial Index - used for spatial data (like coordinates).

18.3 Create Index

Example 1: Create a normal index on department column

```
CREATE INDEX idx_department
ON employees (department);
```

Result:

index_name	column_name	non_unique
idx_department	department	1

Example 2: Create a unique index on email column

```
CREATE UNIQUE INDEX idx_email
ON employees (email);
```

Result:

index_name	column_name	non_unique
idx_email	email	0

Example 3: Create a composite index on department and salary

```
CREATE INDEX idx_dept_salary
ON employees (department, salary);
```

Result:

index_name	column_name	non_unique
idx_dept_salary	department	1
idx_dept_salary	salary	1

18.4 View Indexes

- We can use the SHOW INDEXES command to see all indexes of a table.

```
SHOW INDEX FROM employees;
```

Result:

Table	Non_unique	Key_name	Column_name	Index_type
employees	0	PRIMARY	id	BTREE
employees	1	idx_department	department	BTREE
employees	0	idx_email	email	BTREE
employees	1	idx_dept_salary	department	BTREE
employees	1	idx_dept_salary	salary	BTREE

18.5 Drop Index

- We can remove an index using DROP INDEX.

Example 1: Drop normal index

```
DROP INDEX idx_department
ON employees;
```

Example 2: Drop unique index

```
DROP INDEX idx_email
ON employees;
```

18.6 When to Use Indexes ?

- Columns used frequently in WHERE clause.
- Columns used in JOIN conditions.
- Columns used in ORDER BY and GROUP BY.
- Columns with high cardinality (many distinct values).
- Avoid indexing columns with very few distinct values (like gender).

18.7 Best Practices

- Use indexes on columns that are frequently searched.
- Use composite indexes when querying multiple columns together.
- Avoid creating too many indexes (increases write overhead).
- Choose appropriate column order in composite indexes.
- Regularly monitor and optimize indexes.

Rahul Yadav

19. Transactions in MySQL

19.1 What is a Transaction ?

- A transaction is a sequence of SQL operations that are executed as a single unit of work.
- Either all operations are completed successfully (COMMIT) or none are applied (ROLLBACK).
- Transactions help maintain data consistency and integrity.
- Used in scenarios like fund transfer, order booking, inventory management, etc.

19.2 ACID Properties

Atomicity	All operations in a transaction are completed or none.
Consistency	Database remains in a consistent state before and after.
Isolation	Transactions are isolated from each other.
Durability	Once committed, changes are permanent even in case of system failure.

19.3 Basic Transaction Syntax

```
START TRANSACTION;
-- SQL statements
COMMIT;     -- to save changes
-- or
ROLLBACK;   -- to undo changes
```

Key Points:

- Use START TRANSACTION to begin.
- Use COMMIT to permanently save changes.
- Use ROLLBACK to undo changes.
- If any error occurs, you can rollback.
- Transactions are connection specific.

19.4 Example 1: Fund Transfer between Two Accounts

```
START TRANSACTION;
-- Deduct amount from sender
UPDATE accounts
   SET balance = balance - 1000
   WHERE id = 1;

-- Add amount to receiver
UPDATE accounts
   SET balance = balance + 1000
   WHERE id = 2;
COMMIT;
```

If any error occurs:

```
START TRANSACTION;
UPDATE accounts
SET balance = balance - 1000
WHERE id = 1;

-- some error occurs
ROLLBACK;  -- No changes will be saved
```

19.5 Example 2: Handling Error using TRY-CATCH (in application)

```
START TRANSACTION;
UPDATE orders SET status = 'CONFIRMED' WHERE id = 101;
UPDATE order_items SET status = 'CONFIRMED' WHERE order_id = 101;
-- If any step fails
-- ROLLBACK;
COMMIT;
```

Result:

- If both updates succeed → data saved.
- If any update fails → rollback, no partial data.

19.6 Transaction Isolation Levels

Level	Description	Issues Prevented
READ UNCOMMITTED	Can read uncommitted changes from other transactions.	None (allows dirty reads)
READ COMMITTED	Reads only committed changes.	Prevents dirty reads
REPEATABLE READ	Same data is read within a transaction.	Prevents dirty reads, non-repeatable reads,
SERIALIZABLE	Highest isolation level, transactions run serially.	Prevents dirty reads, non-repeatable reads, phantom reads

19.7 When to Use Transactions ?

- When multiple related operations need to succeed or fail together.
- In banking, e-commerce, order processing, inventory management.
- When maintaining data consistency is critical.

19.8 Best Practices

- Keep transactions short and avoid long-running transactions.
- Use appropriate isolation level based on requirement.
- Always handle exceptions and rollback if needed.
- Avoid performing heavy operations (like large SELECT) inside a transaction.
- Use indexes to reduce locking and improve performance.

Rahul Yadav